Resumen Unidad 3

Creado	@20 de enero de 2026 1:02
Etiquetas	

BBDD Orientadas a clave - valor

¿Qué es redis?

Redis es una base de datos de alto rendimiento in-memory optimizada para ejecutarse en memoria RAM en vez de en el alamacenamiento principal.

Tiene una Cache layer orientada como capa intermedia para sistemas de alta demanda compuesta por una caché LRU (caché de ultima vez usada).

Además sigue un sistema de comunicación mediante publicación y/o subscripción de mensajes

Persistencia y Almacenamiento

Persistencia

- Redis ofrece **durabilidad de los datos opcional**.
- Las versiones disponibles realizan **volcados periódicos de la información al disco duro**.

Almacenamiento de la base de datos

Redis puede almacenar los datos en:

- **Memoria principal (RAM)**
- **Memoria principal + memoria virtual (disco duro)**

Arquitectura y Escalabilidad

Clústeres (Redis v3.x o superior)

- Información **distribuida entre varios nodos**.
- Adecuado para **bases de datos de gran tamaño**.
- **Alta tolerancia a fallos** gracias a la replicación.
- Resistente ante **caídas parciales del clúster**.

Replicación

- Diseñado para arquitecturas **master-slave**.
- Arquitectura **en forma de árbol**:
  - Cada nodo actúa como **master** de las ramas que genera.
  - Cada nodo es **slave** de los nodos anteriores.

Modelo de Datos

Uso de pares clave-valor

Redis almacena la información mediante **claves únicas** asociadas a un **valor**.

Tipos de datos soportados (valores)

- Cadenas de texto
- Cadenas binarias (números)
- Listas
- Conjuntos
- Conjuntos ordenados
- Tablas hash
- Situaciones / coordenadas (geoespaciales)

Casos de Uso

Redis como Base de Datos

Usado cuando se requiere **acceso rápido y frecuente a la información**.

Almacenamiento de sesiones

- La sesión se guarda como **un único objeto**.
- Posibilidad de establecer **tiempo de expiración**.
- Acceso frecuente a los datos de sesión.

Perfiles de usuario y preferencias

- El perfil se almacena en **un solo objeto**.
- Acceso frecuente a la información.

Cesta de la compra

- La cesta se almacena en **un único objeto**.
- Información accedida de forma frecuente.

Redis como Caché

- No es necesario **serializar los datos**, ya que Redis soporta estructuras de datos nativas.
- Almacenamiento de:
  - Datos más consultados del sistema.

- Páginas web más frecuentadas.

Redis como Sistema de Mensajería

- Sistemas de **procesamiento distribuido** mediante colas de mensajes.
- Sistemas de comunicación en tiempo real:
  - Chats
  - Timeline de aplicaciones sociales
  - Mensajes en videojuegos multijugador
  - Sistemas de seguimiento en tiempo real

Ejercicios de Redis — soluciones y comandos

Este documento contiene una breve definición por cada tipo de dato de Redis y la solución de los ejercicios originales en forma de comandos `redis-cli` con la salida esperada (concisa). Está pensado como apuntes y ejemplos reales.

---

1) Claves y Cadenas (Strings)

Definición breve: valores binarios asociados a una clave. Son el tipo más simple: pueden usarse como texto, números (operaciones atómicas) o blobs.

Ejemplos (comandos y salida esperada):

Guarda y sobrescribe

Enunciado: 1. Guarda el nombre de un usuario en user:1001:name con “Ana”, léelo y luego sobrescríbelo con “Ana López”.

```
SET user:1001:name "Ana"
-> OK
GET user:1001:name
-> "Ana"
SET user:1001:name "Ana López"
-> OK
GET user:1001:name
-> "Ana López"
```

Crear solo si no existe y expiración

Enunciado: 2. Crea session:abc solo si no existe y que expire a los 30s; intenta crearlo de nuevo y no debe sobrescribir.

```
SETNX session:abc "data"
-> 1      (1 = creado)
EXPIRE session:abc 30
-> 1
SETNX session:abc "otra"
```

```
-> 0      (no sobrescribe)
GET session:abc
-> "data"
```

TTL, PERSIST y EXPIREAT (timestamp futuro en segundos desde epoch)

Enunciado: 3. Crea temp:key, ponle expiración 10s, comprueba cuánto tiempo le queda para expirar, elimina expiración, vuelve a poner expiración a un timestamp futuro.

```
SET temp:key "x"
EXPIRE temp:key 10
TTL temp:key
-> (por ejemplo) 9
PERSIST temp:key
-> 1
EXPIREAT temp:key 1730000000
-> 1
```

Contador y operaciones con enteros/float

Enunciado: 4. Implementa un contador en page:views, súmale 10, réstale 3 y añade 0.5.

```
SET page:views 0
INCRBY page:views 10
-> (valor) 10
INCRBY page:views -3
-> 7
INCRBYFLOAT page:views 0.5
-> "7.5"
```

Reiniciar contador devolviendo valor anterior atómicamente

Enunciado: 5. Reinicia un contador devolviendo el valor anterior atómicamente.

```
GETSET counter:session 0
-> "7.5"   (valor anterior devuelto)
GET counter:session
-> "0"
```

Múltiples preferencias y evitar sobrescrituras

Enunciado: 6. Inserta varias preferencias (pref:lang, pref:theme y pref:tz) y recupéralas; inserta unas nuevas pref:lang y pref:new evitando sobreescritura si existen.

```
MSET pref:lang "es" pref:theme "dark" pref:tz "Europe/Madrid"
MGET pref:lang pref:theme pref:tz
-> 1) "es" 2) "dark" 3) "Europe/Madrid"
SETNX pref:lang "en"
-> 0   (no sobrescribe)
```

```
SETNX pref:new "valor"
-> 1
```

Comprobar existencia y borrar claves

Enunciado: 7. Comprueba si existe pref:new; borra pref:theme y pref:tz.

```
EXISTS pref:new
-> 1
DEL pref:theme pref:tz
-> 2
```

Log sencillo (listas o strings concatenados)

Enunciado: 8. Construye un log sencillo en log:sys añadiendo líneas.

```
LPUSH log:sys "Inicio sesión: user:1001"
LPUSH log:sys "Acción: ver página"
LRANGE log:sys 0 -1
-> 1) "Acción: ver página" 2) "Inicio sesión: user:1001"
```

---

2) Listas (Lists)

Definición breve: listas ordenadas de strings. Soportan inserción/lectura desde ambos extremos, indexado y recorte.

Crear y mostrar

Enunciado: 1. Crea queue:orders, inserta o1 y o2 por la izquierda, o3 por la derecha. Muestra todo.

```
LPUSH queue:orders o1 o2
-> (longitud)
RPUSH queue:orders o3
LRANGE queue:orders 0 -1
-> 1) o2 2) o1 3) o3   (porque LPUSH añade a la izquierda)
```

Extraer por ambos extremos y longitud

Enunciado: 2. Extrae un elemento por cada extremo y muestra longitud restante.

```
LPOP queue:orders
-> "o2"
RPOP queue:orders
-> "o3"
LLEN queue:orders
-> 1
```

Rellenar y recortar

Enunciado: 3. Rellena list:nums del 1 al 5 y recorta a los tres primeros.

```
DEL list:nums
RPUSH list:nums 1 2 3 4 5
LTRIM list:nums 0 2
LRANGE list:nums 0 -1
-> 1) "1" 2) "2" 3) "3"
```

RPOPLPUSH (cola rotación/transferencia)

Enunciado: 4. Crea src con [a b c], usa RPOPLPUSH src dest y observa dest y src. Describe lo que observas.

```
LPUSH src a b c    # src = [c,b,a]
RPOPLPUSH src dest
-> "a"   (saca de la derecha de src y lo pone a la izquierda de dest)
LRANGE src 0 -1
LRANGE dest 0 -1
# Observación: dest gana el elemento y src pierde ese elemento
```

Modificar índices, insertar y eliminar

Enunciado: 5. Crea list:edit con [x y z]; cambia el índice 1 a Y; inserta new antes de z; elimina todas las x. Intenta LPUSHX sobre una lista inexistente y observa.

```
RPUSH list:edit x y z
LSET list:edit 1 Y
LINSERT list:edit BEFORE z new
LREM list:edit 0 x
LPUSHX nonexist "v"
-> 0   (LPUSHX no crea la lista si no existe)
LRANGE list:edit 0 -1
-> muestra la lista con los cambios
```

---

3) Hashes

Definición breve: mapas campo→valor asociados a una clave. Útiles para objetos/rows pequeños.

Crear y leer

Enunciado: 1. Crea user:2001 con name=Ana, age=30, lee un campo y todo.

```
HSET user:2001 name "Ana" age 30
HGET user:2001 name
-> "Ana"
HGETALL user:2001
-> 1) "name" 2) "Ana" 3) "age" 4) "30"
```

Añadir campos, listar claves y valores

Enunciado: 2. Añade city=Madrid, role=admin; devuelve claves y valores.

```
HSET user:2001 city "Madrid" role "admin"
HKEYS user:2001
-> lista de campos
HVALS user:2001
-> lista de valores
```

Añadir si no existe y borrar

Enunciado: 3. Añade email solo si no existe, comprueba y elimina role.

```
HSETNX user:2001 email "a@example.com"
-> 1 (si no existía)
HGET user:2001 email
-> "a@example.com"
HDEL user:2001 role
-> 1
```

Incrementos (entero y float)

Enunciado: 4. Incrementa score en 5 y luego en 0.75.

```
HINCRBY user:2001 score 5
-> (valor) 5
HINCRBYFLOAT user:2001 score 0.75
-> "5.75"
```

Número de campos y longitud de campo

Enunciado: 5. Muestra el número de campos del hash y longitud del name.

```
HLEN user:2001
-> (ej.) 4
HSTRLEN user:2001 name
-> (ej.) 3
```

Iterar con patrones

Enunciado: 6. Itera por campos que empiecen por c*.

```
HSCAN user:2001 0 MATCH c*
-> devuelve pares campo/valor que empiezan por 'c'
```

---

4) Sets

Definición breve: colecciones no ordenadas de elementos únicos. Operaciones rápidas para pertenencia, unión, intersección y diferencia.

Crear y comprobar pertenencia

Enunciado: 1. Crea tags:post:1 con redis, nosql, db, y comprueba pertenencia de redis.

```
SADD tags:post:1 redis nosql db
SISMEMBER tags:post:1 redis
-> 1
```

Elemento aleatorio sin/eliminando

Enunciado: 2. Obtén un elemento aleatorio sin eliminar y luego elimina otro aleatorio.

```
SRANDMEMBER tags:post:1
-> "redis"   (ej.)
SPOP tags:post:1
-> "nosql"   (ej., elimina uno)
BITCOUNT
```

Diferencia, intersección y unión

Enunciado: 3. s1={a,b,c}, s2={b,c,d}. Calcula diferencia, intersección y unión.

```
SADD s1 a b c
SADD s2 b c d
SDIFF s1 s2
-> {a}
SINTER s1 s2
-> {b,c}
SUNION s1 s2
-> {a,b,c,d}
```

Guardar resultados

Enunciado: 4. Guarda los resultados en sd, si, su y muestra sus miembros.

```
SDIFFSTORE sd s1 s2
SINTERSTORE si s1 s2
SUNIONSTORE su s1 s2
SMEMBERS sd
SMEMBERS si
SMEMBERS su
```

Eliminar y mover

Enunciado: 5. Elimina a de s1 y mueve d de s2 a s1.

```
SREM s1 a
SMOVE s2 s1 d
```

Rellenar y listar por prefijo (usar SSCAN)

Enunciado: 6. Rellena s3 con a1...a5 y b1...b5 y lista solo los que empiecen por a.

```
SADD s3 a1 a2 a3 a4 a5 b1 b2 b3 b4 b5
SSCAN s3 0 MATCH a*
-> lista de elementoss que empiezan por 'a'
```

---

5) Sorted Sets (ZSets)

Definición breve: conjuntos ordenados por score (float). Útiles para rankings.

Crear y mostrar asc/desc

Enunciado: 1. Crea rank:game con (10,"ana"), (20,"juan"), (15,"luis"). Muestra ascendente y descendente.

```
ZADD rank:game 10 "ana" 20 "juan" 15 "luis"
ZRANGE rank:game 0 -1 WITHSCORES
-> 1) "ana" 2) "10" 3) "luis" 4) "15" 5) "juan" 6) "20"
ZREVRANGE rank:game 0 -1 WITHSCORES
-> orden inverso (juan, luis, ana)
```

Rangos por score

Enunciado: 2. Devuelve quienes tienen score entre 12 y 20 (incl.). Luego inverso.

```
ZRANGEBYSCORE rank:game 12 20 WITHSCORES
-> devuelve luis y juan
ZREVRANGEBYSCORE rank:game 20 12 WITHSCORES
-> inverso
```

Establecer todos con score 1 y ejemplo de rango léxico

Enunciado: 3. Pon a todos score 1 y pide de [a a (l (exclusivo).

```
ZADD rank:game 1 "ana" 1 "juan" 1 "luis"
# Para consultas léxicas usar ZRANGEBYLEX en conjunto con members que comp
arten mismo score
```

Incrementar score y ver rango

Enunciado: 4. Suma 5 a “ana”, muestra su puntuación y rango asc/desc.

```
ZINCRBY rank:game 5 "ana"
-> "6"
ZSCORE rank:game "ana"
-> "6"
ZRANK rank:game "ana"
-> (posición ascendente)
ZREVRANK rank:game "ana"
-> (posición descendente)
```

Contar por score y cardinalidad

Enunciado: 5. Cuenta elementos entre score 10 y 20. Cuenta también elementos entre [b y [z. Muestra la cardinalidad de ambos.

```
ZCOUNT rank:game 10 20
-> (nº elementos entre 10 y 20)
ZCOUNT rank:game [b [z   # ejemplo léxico con strings si procede
ZCARD rank:game
-> cardinalidad total
```

Quitar por miembro/posición/score/lex

Enunciado: 6. Quita “juan”. Elimina top-1 por rango de posiciones. Elimina por score <= 12. Elimina por rango léxico [b-[c.

```
ZREM rank:game "juan"
ZREMRANGEBYRANK rank:game 0 0   # elimina top-1 por rango de posiciones
ZREMRANGEBYSCORE rank:game -inf 12
ZREMRANGEBYLEX rank:game [b [c
```

Intersección ponderada (ZINTERSTORE) y Unión (ZUNIONSTORE)

Enunciado: 7. r1: (ana 10, luis 5), r2: (ana 3, luis 20). Interseca ponderando r1*2, r2*1, agrega MAX en r3. 8. Une r1 y r2 en rU utilizando la suma.

```
ZADD r1 10 "ana" 5 "luis"
ZADD r2 3 "ana" 20 "luis"
ZINTERSTORE r3 2 r1 1 r2 WEIGHTS 2 1 AGGREGATE MAX
ZUNIONSTORE rU 2 r1 r2 WEIGHTS 1 1 AGGREGATE SUM
```

---

6) Bits

Definición breve: operaciones a nivel de bit sobre strings; útiles para flags compactos y mapas de bits (bitmaps).

SETBIT/GETBIT

Enunciado: 1. Marca bits 0 y 3 de flags:u1 y consulta sus valores.

```
SETBIT flags:u1 0 1
SETBIT flags:u1 3 1
GETBIT flags:u1 0
-> 1
GETBIT flags:u1 1
-> 0
GETBIT flags:u1 3
-> 1
```

BITOP y BITCOUNT

Enunciado: 2. b1 con bit 0 y b2 con bit 1. BITOP OR, AND, XOR hacia res:* y comprueba con BITCOUNT.

```
SETBIT b1 0 1
SETBIT b2 1 1
BITOP OR res:or b1 b2
BITOP AND res:and b1 b2
BITOP XOR res:xor b1 b2
BITCOUNT res:or
-> número de bits activos en res:or
```

BITPOS (primer bit a 1)

Enunciado: 3. En res:or, encuentra la primera posición con bit 1.

```
BITPOS res:or 1
-> posición (offset en bits)
```

Contar bits en rango de bytes

Enunciado: 4. Cuenta bits activos entre bytes 0 y 1.

```
BITCOUNT key 0 1
-> cuenta bits entre byte 0 y 1
```

---

7) Iteradores (SCAN, SSCAN, HSCAN, ZSCAN)

Definición breve: operaciones de escaneo no bloqueantes para iterar sobre colecciones grandes sin cargar todo en memoria.

SCAN claves

Enunciado: 1. Crea claves user:1...user:5, luego escanéalas con patrón user:*.

```
SET user:1 a
SET user:2 b
SET user:3 c
```

```
SET user:4 d
SET user:5 e
SCAN 0 MATCH user:*
-> devuelve cursors/pares; repetir hasta cursor 0
```

ZSCAN con MATCH

Enunciado: 2. Itera rank:game con MATCH a*.

```
ZSCAN rank:game 0 MATCH a*
-> itera miembros que empiecen por 'a'
```

---

8) Transacciones y WATCH

Definición breve: MULTI/EXEC para agrupar comandos; WATCH permite control de optimistic locking para abortar si hay cambios concurrentes.

1. Encolar y ejecutar

```
MULTI
INCR txn:ct
INCR txn:ct
INCR txn:ct
EXEC
-> lista de resultados (ej. 1,2,3)
```

1. Encolar y descartar

```
MULTI
SET tmp x
DISCARD
-> respuesta vacía y no se aplican comandos
```

1. WATCH (ejemplo read-modify-write seguro)

```
WATCH bal:u1
GET bal:u1
# calcular localmente nuevo = viejo + 10
MULTI
SET bal:u1 <nuevo_valor>
EXEC
-> si alguien cambió la clave entre WATCH y EXEC, EXEC devuelve nil y no s
e aplica
```

---

9) Caché y políticas de desalojo (eviction policies) ← cae

Para limitar memoria y activar política LRU:

```
CONFIG SET maxmemory 10mb
CONFIG SET maxmemory-policy allkeys-lru
```

Nota: en entornos de producción preferible fijarlo en la configuración del servidor redis.conf y verificar el comportamiento con INFO memory.